LETRANDO

Firebase Backend & System Design Specification

MVP Architecture • Authentication • Firestore • Payments • Push Notifications • Security • Observability

Purpose
Defines the backend architecture, data model, integrations, security constraints, operational requirements and implementation sequence for the Letrando mobile literacy application.

Target: Flutter iOS/Android Backend: Firebase/Google Cloud Market: Brazil Language: pt-BR

1. Executive Architecture

Use Firebase as the primary backend platform for the MVP. A separate NestJS backend is intentionally deferred to reduce infrastructure, DevOps, code, cost and attack surface.

```
Flutter
  |
  +-- Firebase App Check
  +-- Firebase Authentication
  +-- Cloud Firestore
  +-- Cloud Storage
  +-- Cloud Functions
  |     +-- Payments
  |     +-- Subscriptions
  |     +-- Notifications
  |     +-- Learning
  |     +-- Admin
  +-- Firebase Cloud Messaging
  +-- Analytics -> BigQuery
  +-- Crashlytics
  +-- Remote Config
  |
  +-- Stripe (initial web checkout)
  +-- Apple App Store / Google Play (future/native billing)
```

Core principle: the mobile client is untrusted. Authorization, payment state, entitlement activation, role assignment and privileged state transitions must be controlled by trusted backend code.

2. Technology Stack

Mobile: Flutter
Auth: Firebase Authentication
Database: Cloud Firestore
Backend: Cloud Functions
Storage: Cloud Storage
Push: FCM
App protection: App Check
Analytics: Firebase Analytics + BigQuery
Crash: Crashlytics
Secrets: Secret Manager
Scheduling: Scheduled Functions / Cloud Scheduler
Payments: Stripe initially; provider abstraction for native store billing.

3. Authentication

Support Email/Password, Google and Apple. Firebase UID is the canonical identity. Providers must be linkable to the same UID.

```
Flutter
  +-- Email/Password --+
  +-- Google OAuth -----> Firebase Authentication -> Firebase UID -> users/{uid}
  +-- Apple OAuth ------+
```

- Never store passwords or third-party credentials in Firestore or source code.
- Use Firebase Auth for email verification, password reset and session management.
- Do not create duplicate accounts when a user links another provider.

4. Firestore Data Model

Private data is user-centric; educational content is global/shared.

```
/users/{uid}
  /children/{childId}
  /devices/{deviceId}
  /subscriptions/{subscriptionId}
  /entitlements/{entitlementId}
  /notifications/{notificationId}
```

```
  /notificationPreferences/{preferenceId}
  /sessions/{sessionId}
  /learningProgress/{progressId}
  /activityAttempts/{attemptId}

/courses/{courseId}
/modules/{moduleId}
/lessons/{lessonId}
/activities/{activityId}
/products/{productId}
/featureFlags/{flagId}
/webhookEvents/{providerEventId}
```

User: email, displayName, locale, timezone, onboarding state, timestamps. Do not store passwords.

```
users/{uid}/children/{childId}
{
  "name": "Lucas",
  "birthDate": "2022-04-15",
  "age": 4,
  "avatarId": "lion_01",
  "learningLevel": "pre_literacy",
  "currentCourseId": "course_001",
  "currentLessonId": "lesson_012",
  "createdAt": "serverTimestamp",
  "updatedAt": "serverTimestamp"
}
```

Child-data constraint: collect the minimum required data. Avoid school, address, precise location, phone number or real child photos in the MVP unless explicitly justified.

```
learningProgress/{lessonId}
{
  "status": "completed",
  "startedAt": "...",
  "completedAt": "...",
  "durationSeconds": 842,
  "score": 87,
  "attempts": 12,
  "correctAnswers": 10,
  "wrongAnswers": 2
}
```

```
activityAttempts/{attemptId}
{
  "activityId": "activity_023",
  "lessonId": "lesson_004",
  "type": "phonics",
  "answer": "BA",
  "correct": true,
  "attemptNumber": 1,
  "responseTimeMs": 1830,
  "createdAt": "serverTimestamp"
}
```

5. Content Architecture

Educational content must be configuration-driven; activities must not be hardcoded into individual screens.

```
Course -> Module -> Lesson -> Activity -> Question -> Answer -> Feedback
```

```
{
  "id": "activity_001",
  "lessonId": "lesson_001",
  "type": "multiple_choice",
  "instruction": "Qual começa com o som B?",
  "audioUrl": "gs://.../b.mp3",
  "options": [
    {"id": "1", "imageUrl": "...", "correct": true},
    {"id": "2", "imageUrl": "...", "correct": false}
  ],
  "difficulty": 1,
  "order": 1
}
```

6. Storage

```
Cloud Storage
  /content
    /audio
    /images
    /illustrations
    /books
    /animations

Firestore stores metadata, relationships and configuration.
Cloud Storage stores binary assets.
```

Never store binary files directly in Firestore.

7. Subscriptions & Entitlements

Keep subscription (commercial/payment relationship) separate from entitlement (authorization to access Premium features).

Founder pricing: Monthly R$19.90 • Quarterly R$49.90 • Annual R$149.90

```
subscription
{
  "provider": "stripe",
  "providerCustomerId": "cus_xxx",
  "providerSubscriptionId": "sub_xxx",
  "planId": "founder_annual",
  "status": "active",
  "currency": "BRL",
  "amount": 14990,
  "interval": "year",
  "currentPeriodStart": "...",
  "currentPeriodEnd": "...",
  "cancelAtPeriodEnd": false
}
```

Store BRL amounts as integer cents: R$149.90 = 14990.

```
entitlement
{
  "active": true,
  "source": "stripe",
  "plan": "founder_annual",
  "features": ["full_curriculum","premium_books","progress_reports"],
  "validFrom": "...",
  "validUntil": "..."
}
```

8. Stripe Integration

Use Stripe Checkout Hosted Page initially. Do not collect card details directly in Flutter.

```
Flutter
 -> Cloud Function: createCheckoutSession
 -> Stripe Checkout
 -> Payment
 -> Stripe Webhook
 -> verify signature
 -> idempotency check
 -> update subscription
 -> update entitlement
```

Critical: never activate Premium from the checkout redirect. The verified webhook is the authoritative payment signal.

```
checkout.session.completed
customer.subscription.created
customer.subscription.updated
customer.subscription.deleted
invoice.paid
invoice.payment_failed

/webhookEvents/{stripeEventId}
{
  "provider": "stripe",
  "eventType": "invoice.paid",
  "processed": true,
```

```
  "processedAt": "..."
}
```

Repeated Stripe events must not cause duplicate side effects.

9. Payment Provider Abstraction

Do not couple the subscription domain directly to Stripe. Define a provider abstraction.

```
PaymentProvider
  +-- StripeProvider
  +-- AppleAppStoreProvider
  +-- GooglePlayProvider
```

Native digital subscriptions distributed through iOS/Android may be subject to Apple/Google billing and store policies. Validate current rules before launch.

10. Push Notifications

Use Firebase Cloud Messaging for daily reminders, achievements, weekly reports and transactional subscription notifications.

```
users/{uid}/devices/{deviceId}
{
  "platform": "ios",
  "fcmToken": "...",
  "appVersion": "1.0.0",
  "deviceModel": "iPhone",
  "osVersion": "18",
  "notificationsEnabled": true,
  "lastSeenAt": "..."
}
```

```
users/{uid}/notificationPreferences/default
{
  "dailyReminder": true,
  "achievement": true,
  "weeklyReport": true,
  "subscription": true,
  "marketing": false,
  "preferredTime": "18:00"
}
```

11. Daily Reminder Logic

A reminder should be sent only when the child has an unfinished daily lesson, notifications are enabled, and the user's local time matches the configured reminder window.

```
unfinished daily lesson?
        |
       YES
        |
notifications enabled?
        |
       YES
        |
within preferred local-time window?
        |
       YES
        |
      FCM
```

Example: “ Hora da nossa aventura de hoje!”

12. Security Architecture

Security is a first-class MVP requirement.

- Firebase Authentication establishes identity.
- Firebase App Check helps verify legitimate app instances.
- Firestore Security Rules enforce parent/child boundaries.

- Cloud Functions handle privileged operations.
- Secrets live in Secret Manager.
- Stripe webhook signatures must be verified.
- Payment processing must be idempotent.
- Privileged roles are assigned server-side.
- Subscription and entitlement state are server-controlled.
- Use least-privilege IAM and auditable administrative access.

```
Request
  -> App Check
  -> Firebase Auth
  -> Security Rules
  -> Firestore

Privileged operation
  -> authenticated/authorized Cloud Function
  -> provider/database
```

Admin SDK/server libraries bypass Firestore Security Rules, so backend code must be protected through IAM and explicit authorization checks.

13. Firestore Authorization Rules

```
Authenticated parent:
  -> read/write own profile
  -> read/write own children
  -> read own subscriptions
  -> read own entitlements
  -> read own notifications/preferences

Authenticated user:
  -> read published educational content

Backend only:
  -> write subscription state
  -> write entitlement state
  -> assign privileged roles

Admin/content editor:
  -> privileged content operations
```

The client must never write subscription.status, entitlement.active, entitlement.validUntil, user.role, payment-provider identifiers or other privileged fields.

14. Roles

parent — normal application user.
content_editor — controlled content management.
admin — operational/administrative access.

Use Firebase custom claims or another server-controlled mechanism. Never allow the client to self-assign a role.

15. Cloud Functions Structure

```
functions/
  auth/
    onUserCreated
  payments/
    createCheckoutSession
    stripeWebhook
    createCustomerPortal
  subscriptions/
    syncEntitlement
  notifications/
    registerDevice
    sendDailyReminders
```

```
    sendNotification
  learning/
    updateProgress
    completeLesson
  analytics/
    trackServerEvent
  admin/
    manageContent
```

Avoid a monolithic function; keep functions narrow and domain-focused.

16. Direct Firestore vs Backend

Direct/controlled Firestore: published content reads, own profile, own progress, validated learning updates.
Backend only: Stripe checkout creation, webhooks, entitlement changes, subscription cancellation, admin operations, notification fan-out.

17. Observability & Analytics

Technical: Crashlytics, Cloud Logging, function errors/latency, webhook failures, notification failures, Firestore operational errors, app version/device distribution.

```
app_opened
onboarding_started
onboarding_completed
child_created
assessment_started
assessment_completed
lesson_started
lesson_completed
activity_started
activity_completed
answer_submitted
daily_goal_completed
subscription_viewed
checkout_started
subscription_started
subscription_renewed
subscription_canceled
payment_failed
notification_sent
notification_opened
```

Learning: accuracy, completion rate, attempts, response time, time per lesson, skill mastery, phoneme/syllable error rate, lesson abandonment.

18. Business Metrics

Acquisition: installs, registrations, onboarding completion.
Activation: first lesson start/completion.
Engagement: DAU, WAU, lessons/user, minutes/day, streak.
Retention: D1, D7, D14, D30.
Monetization: paywall views, checkout starts, purchases, conversion, MRR, ARPU, churn, LTV.
Education: accuracy, completion, attempts, time/activity, difficulty, skill mastery.

19. LGPD & Child Data

The product handles children's educational data and must use privacy-by-design.

- Obtain appropriate parental consent and provide clear privacy information.
- Collect only data necessary for the product.
- Provide deletion mechanisms and define retention periods.
- Avoid unnecessary precise location, school, phone or real child photos.
- Separate parent identity from child learning profile.
- Restrict child data to the owning parent and authorized backend.

• Avoid using child data for marketing without appropriate legal basis and controls.

• Maintain an auditable process for sensitive administrative access.

This is an engineering specification, not legal advice. Obtain Brazilian privacy/legal review before launch.

20. Reliability & Failure Handling

• Payment webhook processing must be idempotent.

• Notification sending must tolerate retries without duplicate user-facing messages.

• Progress updates should be safe to retry.

• Support temporary offline conditions for non-sensitive learning interactions where practical.

• Premium authorization should fail safely if entitlement cannot be verified.

• Use structured logs and bounded retries.

• Configure production backups and disaster-recovery procedures.

21. Environments

```
Firebase projects:
  letrando-dev
  letrando-staging
  letrando-production

Each environment:
  - separate Firestore
  - separate Storage
  - separate Auth configuration
  - separate secrets
  - appropriate Stripe configuration
  - isolated analytics where practical
```

Never use production payment credentials or production data during development.

22. Claude Code Constraints

Put the following rules into the repository CLAUDE.md and treat them as non-negotiable implementation constraints.

```
1. Never trust the mobile client for authorization, subscription status,
   payment status, role assignment, or entitlement activation.
2. Never store passwords or third-party credentials in Firestore/source code.
3. Never hardcode privileged secrets in the mobile app.
4. Verify every Stripe webhook signature.
5. Make every payment webhook handler idempotent.
6. Authorize every privileged operation server-side.
7. Scope every child-data access to the authenticated parent.
8. Never allow the client to mutate entitlement/payment state.
9. Use server timestamps for authoritative timestamps.
10. Store BRL money as integer cents.
11. Prefer small domain-focused Cloud Functions.
12. Test Security Rules and payment state transitions.
13. Do not add NestJS unless an ADR documents the need.
14. Review dependencies for maintenance, security, license and bundle size.
15. Never bypass security rules from client code.
```

23. Implementation Sequence

Sprint 1 — Firebase Foundation: projects, Flutter integration, Auth, Firestore, Storage, App Check, Security Rules, environments.

Sprint 2 — User & Child: parent profile, child profile, multiple children, onboarding, account lifecycle.

Sprint 3 — Learning: courses, modules, lessons, activities, progress, sessions, attempts.

Sprint 4 — Notifications: FCM, device registration, preferences, daily reminder, achievement/lifecycle notifications.

Sprint 5 — Payments: Stripe products/prices, Checkout Session, webhook, subscription state machine, entitlements, customer portal.

Sprint 6 — Observability: Analytics, Crashlytics, Cloud Logging, BigQuery, dashboards, alerts.

Sprint 7 — Security Hardening: Rules tests, App Check enforcement, IAM, secrets, webhook validation, idempotency, audit logs, deletion flows.

24. MVP Acceptance Criteria

- Email/password, Google and Apple auth work on iOS/Android.
- Parent can create multiple child profiles.
- Child data is isolated by parent UID.
- Published content is readable by authenticated clients.
- Premium entitlement cannot be modified by the client.
- Stripe checkout is created only through trusted backend code.
- Stripe webhooks are verified and idempotent.
- Subscription lifecycle updates entitlements correctly.
- FCM tokens are registered and invalid tokens cleaned up.
- Daily reminders respect preferences and timezone.
- Crashlytics and backend logging are active.
- Core product events reach analytics.
- Security Rules have automated authorized/unauthorized tests.
- Secrets are absent from the mobile repository.
- Production and non-production environments are separated.
- Child-data collection follows minimum-data and documented retention/deletion principles.

25. Final Architecture

```
Firebase Project
|
+-- Authentication
|   +-- Email/Password
|   +-- Google
|   +-- Apple
|
+-- Firestore
|   +-- users/{uid}
|   |   +-- children
|   |   +-- devices
|   |   +-- subscriptions
|   |   +-- entitlements
|   |   +-- notifications
|   |   +-- notificationPreferences
|   |   +-- sessions
|   |   +-- learningProgress
|   |   +-- activityAttempts
|   +-- courses / modules / lessons / activities
|   +-- products / featureFlags / webhookEvents
|
+-- Cloud Storage
|   +-- audio / images / books / animations
|
+-- Cloud Functions
|   +-- auth / payments / subscriptions
|   +-- notifications / learning / analytics / admin
|
+-- FCM / App Check / Analytics / Crashlytics / Remote Config
|
+-- External
    +-- Stripe
```

```
+-- Apple App Store (future)
+-- Google Play (future)
```

26. MVP Non-Goals

- No separate NestJS backend.
- No microservices.
- No Kubernetes.
- No AI tutor backend initially.
- No voice-recognition pipeline initially.
- No complex recommendation engine.
- No multiplayer.
- No unnecessary child PII.
- No custom card-entry UI.
- No client-side entitlement authority.

27. Key Architectural Decisions

Firebase-first: fastest managed foundation for MVP.
Firestore: fits parent/child/progress/content data.
Cloud Functions: trusted boundary for payments and privileged operations.
Stripe Checkout: avoids direct card handling in the app.
Subscription ≠ entitlement: decouples payment from authorization.
Provider abstraction: supports future App Store/Play billing.
FCM: push infrastructure for reminders/lifecycle.
App Check: app-integrity layer in addition to Auth.
Analytics + BigQuery: supports product, monetization and educational analysis.
Minimum child data: privacy-by-design.

End of specification.
Changes to authentication, payment authority, entitlement logic, child-data boundaries or security architecture should be documented as explicit Architecture Decision Records (ADRs).